Training the neural network

for image classification!

① Libraries used

→ Cv2 – numpy – tensorflow
seaborn – matplotlib – scikit learn
zipfile – os (maybe)

→ sklearn.preprocessing → MinMax scaler
sklearn.model selection → train-test split

② Extracting images, pixels & classes

② Preprocessing

→ for img_path in files:
    image = Cv2.Imread(image_path)

→ Img_resized = Cv2.resize(Image, (w,h)) // w,h = 128,128
Img_gray = Cv2.CvtColor(Img_resized, cv.ColorBGR2GRAY)

// turn image into single dimensional vector for training neural network

→ Img_flat = Image_gray.ravel()

Ls Images . append (Image_flat)

③ Class_Creation (In loop)

// Changes depending on data set provided

my Case

img_name = os.path.basename (img_path)

if img_name.startswith('b'):

Class_name = 0] → bart

else:

class_name = 1] → homer

// this step is Creating a class to tag each image and name them to keep track and easily train the neural network on the data.

4. normalising the data + train_test_split

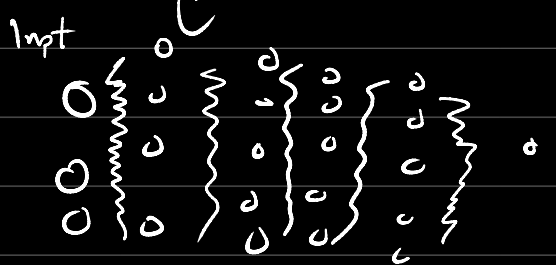
sklearn.preprocessing → MinMax Scaler

→ model_fit (x) // x= model_fit (x)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

5- define neural network (input, hidden, outer_layers)

Network1 = tf.keras.Sequential() ① define network



layers Core in sequence

network.add(tf.keras.layers.Dense(input_shape=(in), units=))

(i) add to network // a layer type= input

hidden layer neurons

n. hidden layer = $\frac{input+2}{2}$

in= 16384, hidden n.= $\frac{16384+2}{2}$ = 8193

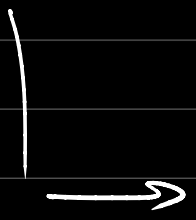
network.add(tf.layer.Dense(units=—, activation='relu')) 1/2

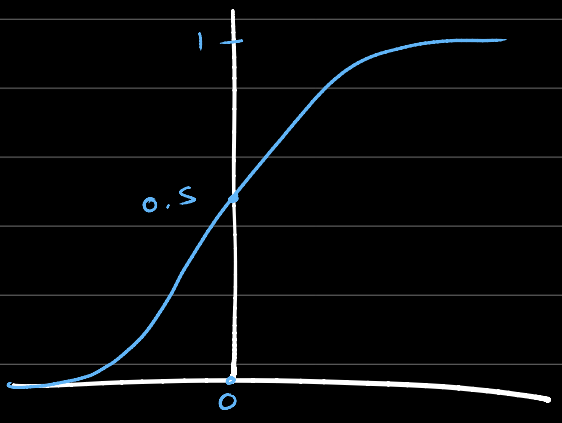
(ii) re-enter units & activation function → defines hidden layer

network.add(tf.layer.Dense(units=—, activation='sigmoid')) 1/3

(iii) re-enter units & activation function → defines outer layer

sigmoid activation function





Sigmoid function

$g(a) = \frac{1}{1+e^{-x}}$

x high → $y \approx 1$

x low → $y \approx 0$

$y > 0$

for outer layer

↓

for output

⑥ Compile & train

network1.compile(optimizer='Adam', loss='binary_crossentropy', metrics=['accuracy'])

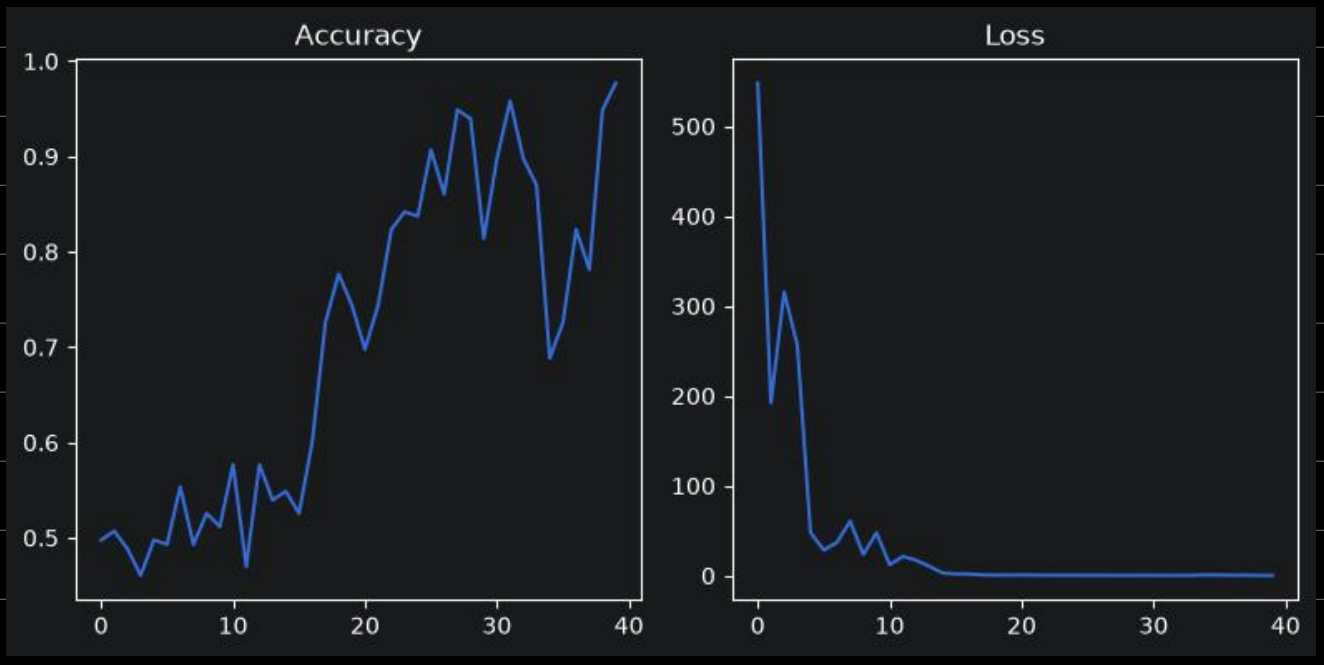
⇒ MyNetwork = network1.fit(x_train, y_train, epochs=40)

→

```
In [37]: history = network1.fit(x_train, y_train, epochs = 40)

Epoch 1/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 10s 1s/step - accuracy: 0.4977 - loss: 547.7338
Epoch 2/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.5070 - loss: 192.2514
Epoch 3/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.4884 - loss: 315.5990
Epoch 4/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.4605 - loss: 256.1954
Epoch 5/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.4977 - loss: 47.7583
Epoch 6/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 9s 1s/step - accuracy: 0.4930 - loss: 28.3845
Epoch 7/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 8s 1s/step - accuracy: 0.5535 - loss: 36.8103
Epoch 8/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 9s 1s/step - accuracy: 0.4930 - loss: 60.5258
Epoch 9/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 9s 1s/step - accuracy: 0.5256 - loss: 23.5688
Epoch 10/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 9s 1s/step - accuracy: 0.5116 - loss: 47.5296
```

→ trains neural network on 40 steps of adjusting weights, measuring loss and accuracy.



→ data Analysis work Afterwards & turning network & weights into usable model

In [81]:
```
modeljs = network1.to_json()
with open('MYFIRSTNETWORKEVER.json', 'w') as json_file:
    json_file.write(modeljs)
```

In [80]:
```
from keras.models import save_model
network1saved = save_model(network1, 'MYFIRSTNETWORKEVER_weight.hdf5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.sav
e_model(model)`. This file format is considered legacy. We recommend using instead the native
Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_mode
l.keras')`.
```

loading saved files

In [82]:
```
with open('MYFIRSTNETWORKEVER.json') as json_file:
    json_saved_model = json_file.read()
json_saved_model
```

In [83]:
```
network1_loaded = tf.keras.models.model_from_json(json_saved_model)
network1_loaded.load_weights('MYFIRSTNETWORKEVER_weight.hdf5')
network1_loaded.compile(loss= 'binary_crossentropy', optimizer='Adam', metrics=['accuracy'])
network1_loaded.summary()
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 8193)	134,242,305
dense_4 (Dense)	(None, 8193)	67,133,442
dense_5 (Dense)	(None, 1)	8,194

```
Total params: 201,383,941 (768.22 MB)
Trainable params: 201,383,941 (768.22 MB)
Non-trainable params: 0 (0.00 B)
```

⇒ this exports network & weights, and makes them reusable!